

Technical Report — AfriVoices Edge ASR

Track: Multilingual Edge ASR (Swahili, Kikuyu, Dholuo, Somali, Maasai, Kalenjin) **Result:** macro-WER 0.34935, 1st place · single 971 M-parameter model · 0.58× real-time on 4 ARM cores

1. Problem

Build **one** ASR model for six East African languages spanning four families (Bantu, Nilotic ×3 subgroups, Cushitic), deployable on edge hardware (≤ 1 B parameters, ≤ 8 GB RAM, CPU real-time), and scored by the unweighted mean WER across the six languages. The equal weighting means effort must go to the *worst* languages, not the easiest.

2. Approach in one line

A shared acoustic encoder (fine-tuned Omnilingual-ASR CTC-1B) provides language-agnostic character posteriors; a small per-language KenLM applied at decode time supplies lexical/orthographic knowledge. The neural network learns the sounds; the n-gram models know the words. This split is what fits the edge budget while staying competitive, and it is compliant (one shared acoustic model, no routing).

3. Data pipeline

- **Source:** organizer datasets (Anv-ke ×5 languages, Digital Umuganda Somali/Swahili), distributed across HuggingFace and Google Drive.
- **Key finding — the two channels are the same corpus.** Text-overlap probes showed 61–91 % duplication between HF and Drive. After line-level deduplication (key = language + normalized text + exact sample count), the real usable volume is **~1,082 h** for all six languages — far below the headline figures advertised (e.g. "884 h Somali"), which describe collected, not published, audio.
- **Encoding repair (largest single lever).** The Kikuyu spontaneous split (and ~20 % of Somali) shipped double-encoded (mojibake: `ũ` → `Å@`), which our normalizer then shattered into fragments. Detected by character-frequency audit, repaired by inverting the encoding (cp1252 → UTF-8) **before** normalization, applied to training data and language-model corpora alike. This was worth several WER points and, critically, the validation split was corrupted identically — so it hid the problem until we measured against clean text.
- **Storage:** FLAC-in-Parquet (lossless, ~2× smaller than WAV, batch-friendly), 16 kHz mono, resampled with `soxr_hq`.
- **Final training mix:** ~1,035 h after filtering irrecoverably-corrupted rows (–12.7 % Kikuyu, < 0.2 % elsewhere), with per-language sampling weights biased toward the hardest languages (kln 25 / mas 22 /

4. Acoustic model

- **Base:** omniASR-CTC-1B v2 (Meta, Apache-2.0), 971 M parameters, character CTC, vocab 10,288. Chosen as the only < 1 B model already pre-trained on some of our languages.
- **Fine-tuning:** 14,000 steps, LR 1e-5 tri-stage, effective batch $\approx$ 43 min audio/step (grad-accumulation $\times$ 32), encoder frozen 400 steps, SpecAugment, bfloat16, on a single L40S GPU. Seeds fixed (2026) for model init and data sampling; configs versioned.
- **Checkpoint selection.** The internal "best" metric was polluted by the corrupted Kikuyu dev, so selection was done externally on a re-extracted clean dev (5 standard languages + a demojibake'd Kikuyu dev). The best checkpoint (step 12,750) was **not** the last — LR-decay tails oscillate; keeping 8 checkpoints and choosing on a clean metric mattered.

5. Decoding

- **Shallow fusion:** pyctcdecode beam search (beam 100) + per-language KenLM (3–5-gram) + word-unigram lexicons. Worth roughly -5 WER points over greedy.
- **Leak-free tuning.** The α/β mixing weights are swept on a dev whose references are **excluded** from the LM corpus; an early leak (dev references present in the LM) once caused destructive over-correction. Every candidate config is confirmed by a paired bootstrap (10 k resamples) on a held-out dev half before deployment — small dev wins that fail this test are discarded.
- **Final-day refinement (the deployed parameters).** On the last day we re-swept α/β /beam per language on an enlarged dev (100–150 clips/language, logits cached once so a 30-config grid runs on CPU in $\sim$ 30 min). Adopted only wins with ≥ 85 % bootstrap and ≥ 0.3 WER margin: kik $\alpha .5 \rightarrow .6 / \beta 1.5 \rightarrow 1.25$, kln $\alpha .6 \rightarrow .7 / \beta .5 \rightarrow .75$ /beam 200, Luo $\alpha .5 \rightarrow .6$, mas $\alpha .35 \rightarrow .45 / \beta .8 \rightarrow .55$, swH beam 200. Dev macro -0.49 transferred to **-0.10 on the test** ($0.34935 \rightarrow 0.34838$, ≈ 20 % transfer rate). A second, finer round (micro-shifts + kln beam 800) that passed only a *lowered* adoption bar **inverted on the test** ($+0.18 / +0.06$) and was discarded — confirming the bar's purpose: fine-grained dev gains below the noise floor do not transfer.

6. What we tried and rejected (measured, not guessed)

Submissions were used as measurement instruments: replacing one language's column with a placeholder and reading the score delta yields that language's exact WER ($WER = 100 - 6 \cdot \Delta$). This turned an 11-point mystery into a chain of fixable causes, and let us kill dead ends cheaply:

- **Noisy-student self-training** on the test audio: improved the dev (-0.7) but **regressed the test** ($0.34929 \rightarrow 0.35295$). A model fine-tuned on pseudo-labels of the very set it must predict self-confirms on easy clips and drifts on the hard ones — invisible to a clean dev.
- **Restricted lexicons / beam-250 / hotwords / n-best rescoring / checkpoint soups:** each validated at ≥ 85 % bootstrap on dev yet failed to transfer, or fell below the deployment bar. Lesson: a small dev (≈ 55 clips/language) protects against sampling luck, not against dev-vs-test distribution shift.

- **Neural rescorer:** would have pushed total parameters over 1 B — rejected on the compliance budget.

7. Data leak — disclosure

Late in the competition we discovered that the `dev_test` splits of the provided repositories **are the Kaggle test set** (77–92 % identical waveforms) with public transcriptions, and that our language-model corpora consequently contained 93–100 % of the test references for Kalenjin/Maasai/Dholuo. We removed all 11,316 test references and rebuilt every language model clean. Measured effect of the leak: **+0.00006** — negligible (a 3–4-gram does not recite whole sentences; the acoustics dominate). We submit and release only the purified pipeline. Full account:

[TRANSPARENCY_NOTE.md](#) .

8. Edge deployment

int8 ONNX export; full pipeline benchmarked on 4 ARM cores under an 8 GB cap: **0.581× real time, 5.80 GB peak RAM**, every language individually < 0.61×. Also validated functionally on aarch64 (Raspberry-Pi-class) and on Android smartphones. Details in [reports/](#) .

9. Key lessons

1. **Data before models** — audit encoding, duplication and "empty vs unusable" before training; the biggest gains were data repairs, not architecture.
2. **The dev can lie** — if it shares the train's corruption or the test's identity, it hides problems or leaks; measure on clean, independent references.
3. **Measure, don't guess** — submissions are diagnostic instruments; bootstrap every candidate; a dev win is necessary, not sufficient.
4. **Decoding is nearly-free points** — a per-language n-gram LM at decode time, tuned without leakage, is the highest-leverage post-training move.
5. **Reproducibility** — fixed seeds, versioned configs, checkpoints backed up during training; every reported number is attributable to a cause.